Drawing the Mona Lisa with Quantum Supersampling

Rintaro Kanaji (6914500223)*

Abstract

We apply Quantum Supersampling (QSS), introduced in Chapter 11 of *Programming Quantum Computers*, to a photograph of the Mona Lisa. The painting is resized to 512×512 subpixels, dithered to one bit and split into 128×128 tiles of 4×4 subpixels. Each tile is drawn into the phases of a four-qubit register by a phase oracle. Its brightness is then estimated by quantum sum estimation, that is, controlled Grover iterations followed by an inverse quantum Fourier transform (QFT). We build all circuits at the gate level in Qiskit, derive a closed form for the QSS lookup table, and find that it agrees with the simulation to within 10^{-11}. We compare QSS with Monte Carlo supersampling at the same number of shader calls. With four counter qubits and a single readout per tile, the setting of the book's lookup-table example, QSS lowers the mean pixel error by 21% and almost doubles the fraction of exactly correct pixels (50.8% versus 28.6%). QSS concentrates its error into fewer but larger, isolated mistakes, which reproduces the book's main observation that a QPU can change the *character* of sampling noise.

1 Introduction

In ray tracing, each output pixel is the average of many samples (rays), a procedure called *supersampling* or Monte Carlo sampling. Only the *sum* of the samples is needed, not the samples themselves. Chapter 11 of the textbook [1] observes that a QPU can estimate such a sum directly. It combines three primitives for this purpose: quantum arithmetic (Ch. 5), amplitude amplification (Ch. 6) and the QFT (Ch. 7). The resulting algorithm, QSS, is not faster than Monte Carlo sampling, but its error is distributed differently: most pixels come out exact, and a few are badly wrong.

Throughout, the *shader* is the function $f(x) \in \{0, 1\}$ that returns the value of subpixel x. On a QPU the shader is applied as a phase oracle, and the cost of every method is counted in *shader calls*. The book demonstrates QSS on synthetic shapes such as lines, circles and circular bands, whose shaders are short arithmetic circuits. This report applies the same pipeline to a photograph, which needs a lookup-table shader instead. We cover the image preparation, the circuits, the mathematics of the readout, gate-level verification, a comparison with Monte Carlo sampling at equal cost, confidence maps and colour output. A public implementation of a similar pipeline [5] was consulted for orientation; the code and analysis here are our own.

*Homework report for *Programming Quantum Computers*, Chapter 11.

2 Image preparation

The input is a 408×530 RGB image. We crop a 408×408 square starting at row 20 so that the head is kept, then resize it to 512×512 with Lanczos filtering. A 4×4 Bayer matrix

$$B = \begin{pmatrix} 0 & 8 & 2 & 10 \\ 12 & 4 & 14 & 6 \\ 3 & 11 & 1 & 9 \\ 15 & 7 & 13 & 5 \end{pmatrix} \tag{1}$$

is aligned with the tile grid. A subpixel of grey level $g \in [0, 1]$ at position (u, v) is set white ($b_{uv} = 1$) when

$$g_{uv} \geq \frac{B_{u \bmod 4,\, v \bmod 4} + \frac{1}{2}}{16}. \tag{2}$$

Each threshold occurs exactly once per tile, so a tile of uniform grey level g contains exactly $k = \lfloor 16g + \frac{1}{2} \rfloor$ white subpixels. The number of white subpixels is therefore a faithful 17-level measure of brightness, and the *ideal* output pixel is

$$I^{\star}_{ij} = \frac{k_{ij}}{16}, \qquad k_{ij} = \sum_{(u,v) \in T_{ij}} b_{uv}, \tag{3}$$

where T_{ij} is the tile in row i and column j. For colour, the R, G and B channels are dithered independently into three bitplanes. Figure 1 shows the stages.

3 Phase-encoded tiles

A tile is addressed by two 2-qubit registers q_x, q_y, combined into the 4-qubit index $x = 4q_y + q_x \in \{0, \ldots, 15\}$. The shader $f(x) = b_{uv}$ returns the dithered bit of the subpixel (u, v) of the tile that has index x. Hadamard gates prepare a blank canvas, and the phase oracle $O_f|x\rangle = (-1)^{f(x)}|x\rangle$ paints the white subpixels:

$$\begin{aligned} |s\rangle &= H^{\otimes 4}|0\rangle = \frac{1}{4}\sum_{x=0}^{15}|x\rangle, \\ O_f|s\rangle &= \frac{1}{4}\sum_{x=0}^{15}(-1)^{f(x)}|x\rangle. \end{aligned} \tag{4}$$

This is the book's *phase encoding*. All amplitudes have magnitude 1/4, and the image is stored only in the signs (Fig. 2). A photograph has no compact arithmetic description, so we write the oracle as a product of single-state phase flips,

$$O_f = \prod_{x:\, f(x)=1} \left(X^{\bar{x}}\, C^3 Z\, X^{\bar{x}} \right), \tag{5}$$

where $X^{\bar{x}}$ applies X to every qubit whose bit of x is 0, and C^3Z is a Z gate controlled by three qubits, which flips the phase of $|1111\rangle$.

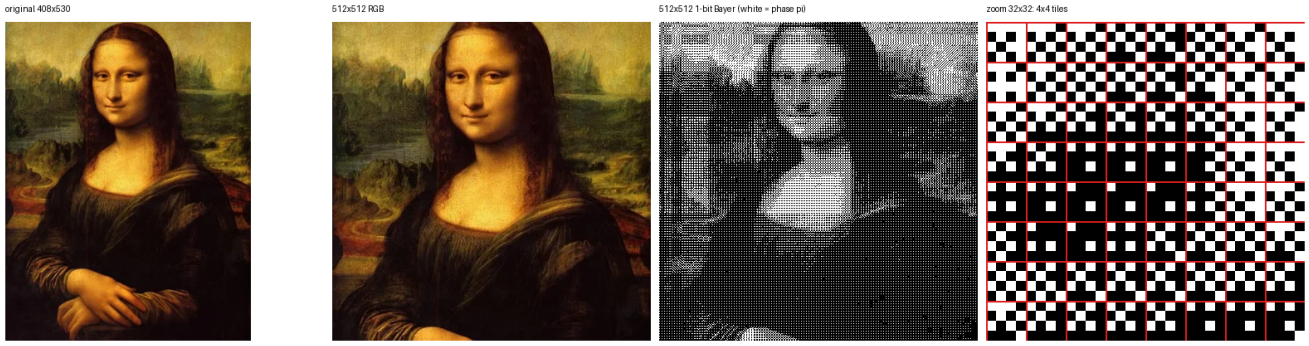


Figure 1: Image preparation. From left: original (408×530), 512×512 crop, one-bit Bayer image (white = phase π), and a 32×32 zoom with the 4×4 tile grid in red. Each tile becomes one output pixel.

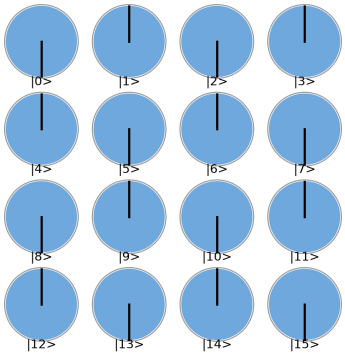


Figure 2: Circle notation of the tile at $(x, y) = (64, 35)$ (a checkerboard, $k = 8$) after $H^{\otimes 4}$ and O_f. A line pointing up means phase 0 (black); a line pointing down means phase π (white).

4 Quantum sum estimation

4.1 Grover iteration

We use the book's `grover_iteration`, which reflects about $|s\rangle$:

$$\begin{aligned} Q &= \left(I - 2|s\rangle\langle s|\right) O_f \\ &= H^{\otimes 4} X^{\otimes 4} C^3 Z \, X^{\otimes 4} H^{\otimes 4} \, O_f. \end{aligned} \tag{6}$$

Define $\theta \in [0, \pi/2]$ by

$$\sin^2 \theta = \frac{k}{16}, \tag{7}$$

and the normalised states $|w\rangle = k^{-1/2} \sum_{f(x)=1} |x\rangle$ and $|b\rangle = (16-k)^{-1/2} \sum_{f(x)=0} |x\rangle$, so that $|s\rangle = \sin\theta|w\rangle + \cos\theta|b\rangle$. On the plane spanned by $|b\rangle$ and $|w\rangle$, O_f is a reflection about $|b\rangle$ and $2|s\rangle\langle s| - I$ is a reflection about $|s\rangle$, so their product is a rotation by 2θ. Because Eq. (6) carries an extra minus sign, Q acts in the basis $(|b\rangle, |w\rangle)$ as

$$Q = -\begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}, \tag{8}$$

with eigenvalues $e^{i(\pi \pm 2\theta)}$ and eigenvectors $|\psi_\pm\rangle$. The start state is an equal-weight superposition of the two,

$$|s\rangle = \tfrac{1}{\sqrt{2}}\left(e^{i\alpha}|\psi_+\rangle + e^{-i\alpha}|\psi_-\rangle\right), \tag{9}$$

where α is a phase that does not affect the measurement probabilities. (For $k = 0$ or 16 the two eigenvalues coincide and $|s\rangle$ is itself an eigenvector.)

4.2 Counter register and inverse QFT

A t-qubit counter in uniform superposition controls Q^{2^j} on its j-th qubit, so counter value m triggers exactly m iterations:

$$\frac{1}{\sqrt{N}} \sum_{m=0}^{N-1} |m\rangle \otimes Q^m |s\rangle, \qquad N = 2^t. \tag{10}$$

Applying $\mathrm{QFT}^\dagger$ to the counter and measuring gives a readout $y \in \{0, \ldots, N-1\}$. For an eigenphase $2\pi\varphi$ the readout distribution is the normalised Fejér kernel

$$F_N(\delta) = \left| \frac{1}{N} \sum_{m=0}^{N-1} e^{2\pi i m \delta} \right|^2 = \frac{\sin^2(\pi N \delta)}{N^2 \sin^2(\pi \delta)}, \tag{11}$$

where $\delta = \varphi - y/N$ and $\sum_y F_N = 1$. With Eq. (9) and $\varphi_\pm = \frac{1}{2} \pm \theta/\pi$, the *QSS lookup table* has the closed form

$$\boxed{P(y \mid k) = \tfrac{1}{2}\left[F_N\!\left(\varphi_+ - \tfrac{y}{N}\right) + F_N\!\left(\varphi_- - \tfrac{y}{N}\right)\right]} \tag{12}$$

for all $0 \le k \le 16$. Three consequences follow. (i) $P(y \mid k)$ depends only on k, not on *where* the white subpixels are. (ii) Its peaks sit at

$$y \approx N\left(\tfrac{1}{2} \pm \tfrac{1}{\pi} \arcsin\sqrt{k/16}\right) \pmod N, \tag{13}$$

so black tiles ($k = 0$) are read near $y = N/2$ and white tiles ($k = 16$) near $y = 0$. (iii) Because $\varphi_+ + \varphi_- = 1$ and F_N is even and 1-periodic, $P(y \mid k) = P(N - y \mid k)$. The readouts y and $N - y$ carry the same information, so at most $N/2 + 1$ distinct values of k can be decoded.

4.3 Decoding and confidence

Book Example 11-5 builds the table. As in Example 11-4 (`qss_count_to_hits`), we decode a readout to the most likely count,

$$\hat{k}(y) = \arg\max_k P(y \mid k), \qquad \hat{I}_{ij} = \hat{k}(y_{ij})/16. \tag{14}$$

Table 1: Maximum absolute deviation in each verification test.

Test	$t = 4$	$t = 6$
(a) gate-level Q vs. matrix	4.6×10^{-15}	1.6×10^{-15}
(b) permuted masks vs. table	9.3×10^{-15}	4.0×10^{-14}
(c) 40 real tiles vs. table	1.9×10^{-14}	6.1×10^{-14}
(d) sim. vs. closed form	1.6×10^{-12}	6.7×10^{-12}

The confidence of a pixel is the posterior probability of the decoded value under a uniform prior,

$$c(y) = \frac{P(y \mid \hat{k}(y))}{\sum_{k=0}^{16} P(y \mid k)}. \tag{15}$$

Figure 3 shows the tables. With $t = 4$, consequence (iii) leaves only nine decodable levels, $\hat{k} \in \{0, 1, 2, 5, 8, 11, 14, 15, 16\}$. Tiles with $k \in \{3, 4, 6, 7, 9, 10, 12, 13\}$ can never be decoded exactly. With $t = 6$ all 17 levels are decodable.

4.4 Circuits

Figure 4 shows the circuit for one tile with $t = 4$, which uses $4 + t = 8$ qubits. The controlled iteration needs only *one* extra control on each C^3Z gate (Fig. 6). When the control is $|0\rangle$, the X and H conjugations cancel in pairs. This is the same idea as the book's `mask_with_condition`. Figure 5 shows the inverse QFT. After decomposition into CNOT and single-qubit gates, a $t = 4$ tile circuit contains about 1.6×10^3 CNOTs and a $t = 6$ circuit (10 qubits) about 6.8×10^3.

5 Implementation and verification

The circuits are built from Qiskit 2.6 gates (H, X, multi-controlled Z, `QFTGate`) and simulated exactly with `Statevector`. We verified the implementation in four ways (Table 1): (a) the unitary of the gate-level Q equals the matrix $(I - 2|s\rangle\langle s|)\,\mathrm{diag}((-1)^f)$; (b) for every k, a mask with its white subpixels randomly permuted gives the same $P(y)$; (c) 40 real tiles of the Mona Lisa give the table column for their own k; (d) the simulated table equals the closed form, Eq. (12).

Tests (b) and (c) show that the readout statistics of a tile are fully determined by its count k. We therefore render the full image by drawing one readout y_{ij} per tile from $P(y \mid k_{ij})$. This is statistically identical to running and measuring each of the 16 384 tile circuits once, and it takes seconds instead of hours.

5.1 Monte Carlo baseline

Conventional supersampling evaluates the shader at n subpixels chosen uniformly at random (with replacement). The number of white hits H follows $H \sim \mathrm{Bin}(n, k/16)$, and the estimate is $\tilde{I} = \mathrm{round}(16H/n)/16$, rounded to the same 17 levels. The circuit of Fig. 4 calls the shader $\sum_{j=0}^{t-1} 2^j = 2^t - 1$ times, so for equal cost we use $n = 2^t - 1$, which is 15 for $t = 4$ and 63 for $t = 6$. The standard error of the Monte Carlo estimate of $p = k/16$ is

$$\sigma_{\mathrm{MC}} = \sqrt{\frac{p(1-p)}{n}} = O(n^{-1/2}). \tag{16}$$

The QSS peak width in Eq. (11) is $O(1/N)$ in φ, and hence also in $p = \sin^2\theta$. This is the quadratic speedup of amplitude estimation [2]. This baseline models a renderer, where the number of possible samples is effectively unlimited. A tile here has only 16 subpixels, so 16 deterministic shader calls would give the exact value classically. The comparison is therefore at a fixed sample budget, as in the book, and is not the best possible classical strategy for this toy problem.

Table 2: Greyscale results at equal shader calls per pixel. Values in parentheses are expectations computed from $w(k)$ and Eq. (12).

Method	Calls	MAE	Exact (%)
QSS, $t = 4$	15	**0.0574** (0.0591)	**50.8** (50.5)
Monte Carlo	15	0.0725 (0.0725)	28.6 (28.1)
QSS, $t = 6$	63	**0.0115** (0.0109)	**91.1** (91.2)
Monte Carlo	63	0.0321 (0.0322)	54.6 (54.3)

6 Results

For any estimate $\hat{I}$ (QSS) or $\tilde{I}$ (Monte Carlo) we report the mean absolute error $\mathrm{MAE} = \frac{1}{|\Omega|}\sum_{ij} |\hat{I}_{ij} - I^\star_{ij}|$ and the fraction of exactly correct pixels $E_{\mathrm{exact}} = \frac{1}{|\Omega|}\#\{ij : \hat{I}_{ij} = I^\star_{ij}\}$. Here Ω is the set of 128×128 pixels. With $w(k) = \#\{ij : k_{ij} = k\}/|\Omega|$, the fraction of tiles with count k, the expected values for QSS are

$$\mathbb{E}[\mathrm{MAE}] = \sum_k w(k) \sum_y P(y \mid k)\, \frac{|\hat{k}(y) - k|}{16}, \tag{17}$$

$$\mathbb{E}[E_{\mathrm{exact}}] = \sum_k w(k) \sum_y P(y \mid k)\, [\hat{k}(y) = k], \tag{18}$$

and similarly for Monte Carlo with $\mathrm{Bin}(n, k/16)$ in place of $P(y \mid k)$. Table 2 lists both the sampled and the expected values, which agree closely.

Error level. In the book's setting ($t = 4$, one readout), QSS lowers the MAE by 21% and gives 1.78× as many exact pixels. The book reports about 33% and about 2× for its synthetic test image. At $t = 6$ the MAE reduction grows to 64%, and the exact-pixel ratio is 1.67× (91.1% versus 54.6%). This is qualitatively consistent with the $O(1/N)$ versus $O(n^{-1/2})$ scaling. Part of the QSS gain from $t = 4$ to $t = 6$, however, comes from going from 9 to 17 decodable levels. In addition, the $1/\delta^2$ tails of the Fejér kernel make the single-readout error scale more like $O(\log N/N)$.

Character of the noise. The error maps in Fig. 7 show the effect described in the book. Monte Carlo error is a fine grain on *every* pixel, largest where $p(1-p)$

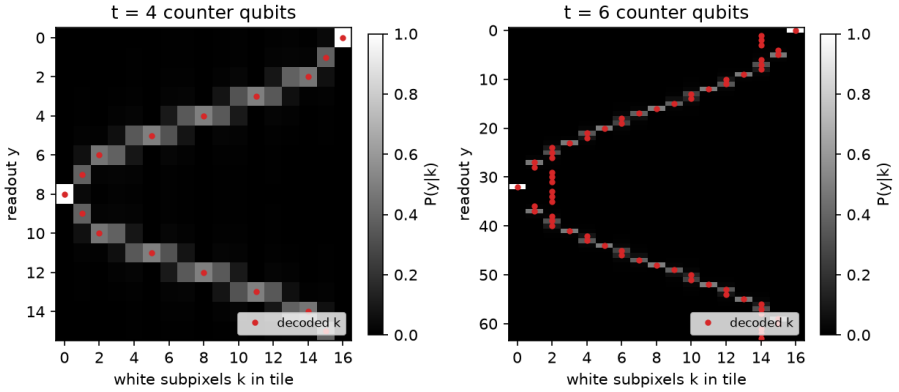


Figure 3: QSS lookup tables $P(y \mid k)$ for 4×4 tiles with $t = 4$ (left) and $t = 6$ (right) counter qubits. Red dots mark $\hat{k}(y)$. Each table is symmetric under $y \leftrightarrow N - y$. With $t = 4$ only nine levels are decodable and neighbouring columns overlap; with $t = 6$ every k has its own peak. Compare book Figs. 11-14 and 11-16.

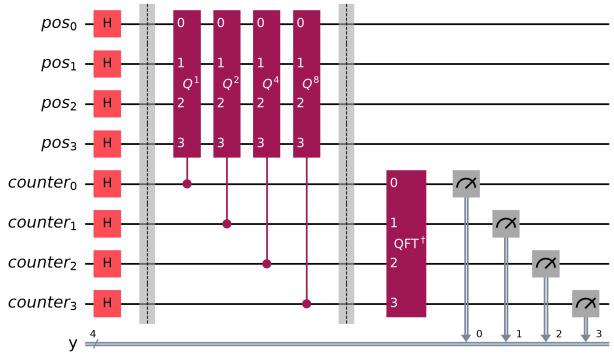


Figure 4: QSS circuit for one tile ($t = 4$). Counter qubit c_j controls Q^{2^j}; $\mathrm{QFT}^\dagger$ converts the counter phases into a readout y.

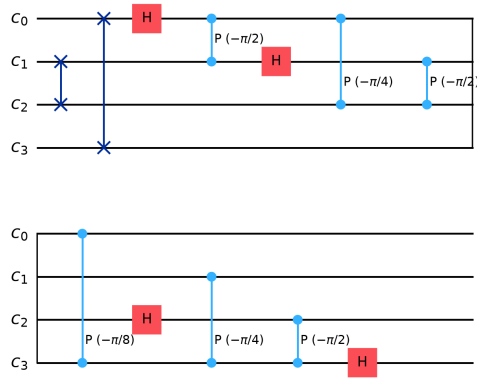


Figure 5: Four-qubit inverse QFT on the counter: bit-reversal SWAPs, Hadamards and controlled phases $P(-\pi/2^\ell)$.

is largest, i.e. at mid-tones. QSS leaves most pixels exactly right and puts the rest of its error into fewer but larger mistakes, up to 0.94 in the worst pixel. At $t = 4$ these comprise small offsets for the eight undecodable mid-tone levels, together with isolated high-contrast specks. At $t = 6$ only 8.9% of pixels are wrong, and these are mostly isolated specks. A speck appears when a readout lands in the slowly decaying tail of the Fejér kernel, which wraps around modulo N. Readouts on the mirror peak $N - y$ are harmless, because they decode to the same k. In the book's words, conventional supersampling *converges toward* the exact answer, while QSS raises the *probability* of the exact answer. Isolated specks are easy to remove afterwards, for example with a median filter or by re-sampling; a uniform grain is not.

Confidence map. Figure 8 shows Eq. (15) for $t = 4$. It is computed from the readouts alone, without knowledge of the true image. Confidence is high for very dark tiles ($y \approx N/2 = 8$) and very bright tiles ($y \approx 0$), and low for mid-tones, where the table columns overlap. The Pearson (point-biserial) correlation between $c(y)$ and actual correctness is 0.47. The map could therefore steer a second pass that re-samples only the low-confidence pixels.

Colour. Following the book's "Adding Color" section, the R, G and B bitplanes are supersampled independently and recombined (Fig. 9). Each channel has 17 levels, giving $17^3 = 4913 \approx 2^{12.3}$ colours per pixel. This matches the book's "effectively 12-bit-per-pixel" image. Over all channels at equal cost ($t = 4$, $n = 15$), QSS has an MAE of 0.050 with 58.7% exact values, against 0.064 and 34.0% for Monte Carlo. At $t = 6$, $n = 63$ the figures are 0.0096 and 92.4% against 0.0285 and 58.9%. A QSS error shows up as a saturated red, green or blue speck: one bitplane is wrong while the other two remain exact.

7 Discussion

What the QPU contributes. The QSS circuit never outputs an individual subpixel value; it estimates $k = \sum_x f(x)$ over all 16 subpixels in superposition. The shader still runs $2^t - 1$ times per pixel, so there is no speed-up in shader calls. The benefit is the shape of the error distribution and its faster decrease with N.

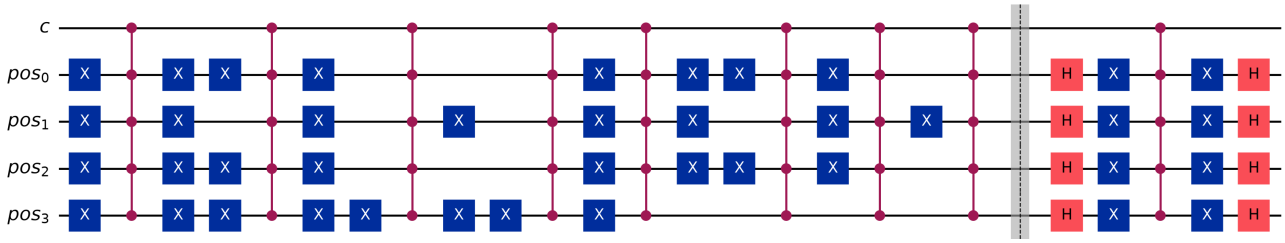


Figure 6: Gate-level controlled-Q for the tile at $(x, y) = (64, 35)$. Left of the barrier: the oracle O_f, made of eight phase flips of the form of Eq. (5). Right: the diffusion $I - 2|s\rangle\langle s|$. Only the multi-controlled Z gates carry the counter control c.

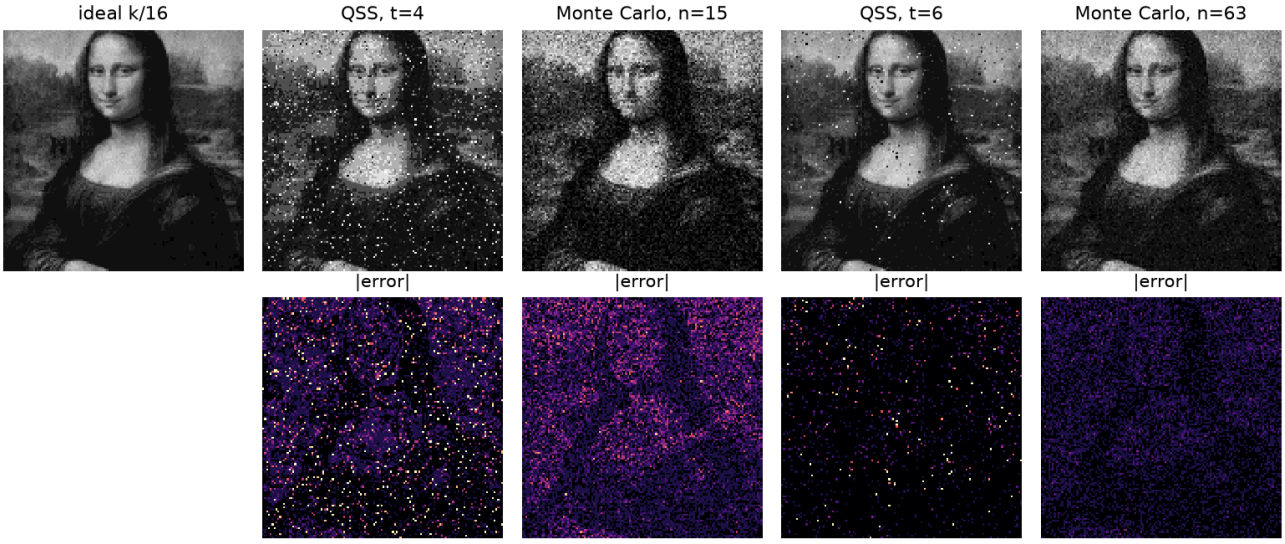


Figure 7: Greyscale results (128×128). Top, from left: ideal (Eq. (3)); QSS with $t = 4$ and Monte Carlo with $n = 15$ shader calls; QSS with $t = 6$ and Monte Carlo with $n = 63$. Each QSS tile uses a single readout. Bottom: absolute error $|\hat{I} - I^{\star}|$.

Ambiguity at $t = 4$. The symmetry $P(y \mid k) = P(N - y \mid k)$ leaves only nine distinguishable outcomes for 17 counts. Moreover, mid-tones are crowded in phase: near $\theta = \pi/4$ the map $k \mapsto \varphi$ is flattest, and φ_+ for $k = 8$ and $k = 9$ differ by only $0.020 < 1/N = 0.0625$. Both effects explain the speckle and the low confidence at mid-tones, and both disappear at $t = 6$.

Cost of a photographic shader. The book's shaders draw shapes with a few arithmetic gates. A photograph has no such formula, so Eq. (5) needs up to 16 multi-controlled gates per oracle call. One tile then needs thousands of CNOT gates, far beyond today's noisy hardware. Compressing the oracle would reduce this cost, for example by exclusive-sum-of-products (ESOP) or Reed–Muller synthesis of f, or by flipping the black subpixels together with a global sign when $k > 8$.

Other image representations. FRQI [3] stores grey levels in a rotation angle, and NEQR [4] stores them in a basis-state register; both hold greyscale images without dithering. As the book notes, putting brightness into amplitudes would break the quantum sum estimation that QSS relies on. We therefore keep a one-bit phase encoding and obtain tone through dithering and bitplanes.

Limitations. All results come from noiseless simulation, and the bitmap is prepared classically. The per-tile readouts are sampled from the lookup table, which is justified by the tests in Table 1. As discussed in the Monte Carlo baseline subsection, the classical baseline uses a fixed budget. The results characterise the algorithm; they do not demonstrate a hardware advantage.

8 Conclusion

We drew the Mona Lisa with the complete QSS pipeline of Chapter 11. The pipeline uses phase-encoded tiles, a lookup-table shader, controlled Grover iterations, an inverse QFT, a lookup table with the closed form Eq. (12), confidence maps and colour bitplanes. At equal sampling cost, QSS gives a lower mean error than Monte Carlo and nearly twice as many exactly correct pixels. Its noise is concentrated in fewer, larger mistakes rather than spread as grain over every pixel. This reproduces the book's central point: a QPU can be useful by redistributing noise, even without being faster.

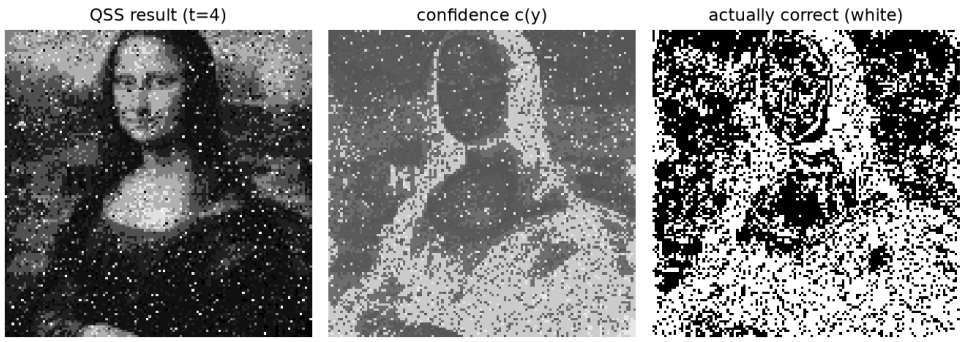


Figure 8: QSS result ($t = 4$), its confidence map $c(y)$ from Eq. (15) (bright = likely correct), and the pixels that are actually correct (white).

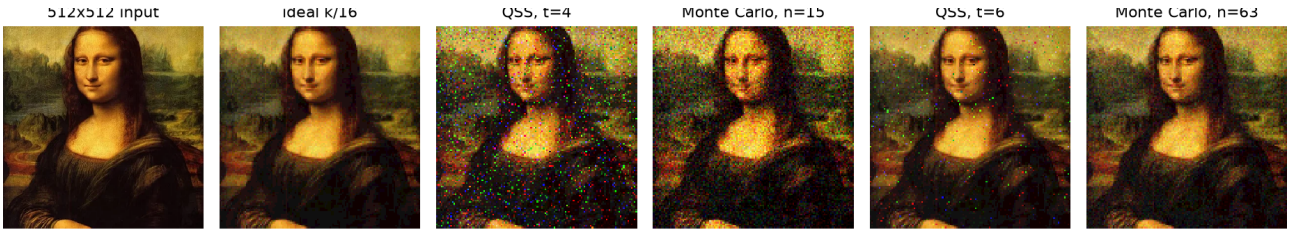


Figure 9: Colour by bitplanes. From left: 512×512 input; ideal; QSS $t = 4$; Monte Carlo $n = 15$; QSS $t = 6$; Monte Carlo $n = 63$.

Code availability

`src/prepare_image.py` prepares the image (Sec. 2). `src/qss.py` builds the circuits, runs the verification including test (d) and the expected values, and draws all figures. It takes about 80 s on a laptop. Numerical results are in `results/metrics.json`.

References

[1] E. R. Johnston, N. Harrigan, M. Gimeno-Segovia, *Programming Quantum Computers*, O'Reilly, 2019, Ch. 11.

[2] G. Brassard, P. Høyer, M. Mosca, A. Tapp, "Quantum amplitude amplification and estimation," *Contemp. Math.* **305**, 53–74 (2002).

[3] P. Q. Le, F. Dong, K. Hirota, "A flexible representation of quantum images," *Quantum Inf. Process.* **10**, 63–84 (2011).

[4] Y. Zhang, K. Lu, Y. Gao, M. Wang, "NEQR: a novel enhanced quantum representation of digital images," *Quantum Inf. Process.* **12**, 2833–2860 (2013).

[5] pongsakornsaks-KU, *quantum-programming*: quantum-supersampling project, GitHub, `https://github.com/pongsakornsaks-KU/quantum-programming`.